Internet and Web Engineering Project

Callegari Luca, Damiano Mirko, Denti Sara

1 Introduction

1.1 File transfer over UDP

This Internet and Web Engineering project implements a C application for transferring files between a client and a server. The application uses the UDP protocol and the Berkeley sockets API.

1.2 Project structure

The project uses three directories, `client`, `server` and `shared`, and a `Makefile` for compilation.

The client directory contains the following files:

- The `client` executable
- `client.c`, the client program's entry point
- `header.c` and `header.h`, which provide supporting functions for the client
- `buffer_client.c` and `buffer_client.h`, which handle buffers
- `SRclient.c` and `SRclient.h`, which implement the Selective Repeat protocol
- `timer_client.c` and `timer_client.h`, which handle timers

The server directory contains the following files:

- The `server` executable
- `SRserver.c` and `SRserver.h`, which implement the Selective Repeat protocol. The server starts in `SRserver.c`.
- `server.c` and `server.h`, which handle file transfers
- `header.c` and `header.h`, which provide supporting functions for the server
- `buffer_server.c` and `buffer_server.h`, which handle buffers
- `timer_server.c` and `timer_server.h`, which handle timers

Finally, the shared directory contains the following files.

- `config.h,` which defines the macros
- `packet.c` and `packet.h`, which handle the packets exchanged between the client and server

2 Packet structure

Packets exchanged between the client and server contain three fields.

```
typedef struct _packet_t {
    packet_header_t header;
    char filename [MAX_FILENAME_LENGTH];
    char body[];
} packet_t;
```

The packet header has the following structure.

```
typedef struct _packet_header_t {
   char syn_flag; // 1 during the first two steps of the three-way handshake, 0
                  // otherwise
   char rst_flag; // 1 if the server cannot establish a new
                  // connection, 0 otherwise
   char fin_flag; // 1 if the sender has finished sending data
   index_t index; // Identifies the established connection

   nseq_t n_seq; // Packet sequence number
   nseq_t ack; // Packet acknowledgement number
   char ack_flag; // 1 if the packet is an ACK, 0 otherwise

   cmd_t cmd; // Identifies the command
   pos_t pos; // Position of the first byte
   char end_flag; // 1 for the final packet in the data stream, 0 otherwise
   char error_flag;// 1 if an error occurred while executing the
                   // command, 0 otherwise

   packet_len_t len;// Packet length
   packet_len_t body_len; // Packet body length

   struct sockaddr_in address;
   // Destination socket address (valid on the server only)
} packet_header_t;
```

2.1 The `make_packet()` function

Both the client and the server call `make_packet()` in `packet.c` to create a packet.

```
packet_t *make_packet(){
   packet_len_t packet_len = sizeof(packet_header_t)+ MAX_FILENAME_LENGTH
   +_body_len;

   // Allocate the packet
   packet_t *packet = malloc(packet_len);
   if (!packet) {
      error(stderr, ``Errore durante l'allocazione␣del␣pacchetto");
   }

   memset(packet, 0, packet_len);
   // Assign the corresponding value to each structure field
   packet->header.syn_flag = _syn_flag;
   packet->header.rst_flag = _rst_flag;
   packet->header.fin_flag = _fin_flag;
```

```
    packet->header.index = htonl(_index);

    packet->header.n_seq = htonl(_n_seq);
    packet->header.ack = htonl(_ack);
    packet->header.ack_flag = _ack_flag;

    packet->header.cmd = htons(_cmd);
    packet->header.pos = htonl(_pos);
    packet->header.end_flag = _end_flag;
    packet->header.error_flag = _error_flag;

    packet->header.len = htonl(packet_len);
    packet->header.body_len = htonl(_body_len);

    memcpy(packet->filename, _filename, _filename_len);
    memcpy(packet->body, _body, _body_len);
    return packet;
}
```

The function allocates memory for the entire packet and initialises its fields. It uses `htons()` and `htonl()` to convert integers from host byte order to network byte order for transmission.

The initial field values are passed as arguments to the function. The arguments are omitted from this listing for readability.

3 Top-down code architecture

The project has two layers, both of which are required for reliable file transfer.

- **Layer 1 — Data transfer (upper layer)**

  The upper layer manages file transfers between the client and server. It handles high-level operations such as splitting files into packets, sending the packets and reassembling the files.

- **Layer 2 — Flow control and reliability (lower layer)**

  The lower layer implements the Selective Repeat protocol using the following mechanisms.

  - **Acknowledgements**: ACK packets confirm successful packet reception.
  - **Timers**: If a packet is not acknowledged within the timeout interval, it is retransmitted or the connection is terminated. The timeout can be fixed or adjusted dynamically as network conditions change.
  - **Sliding window**: The window controls data flow by allowing several packets to be in flight while awaiting acknowledgement, improving network utilisation.

4 Layer 1

Layer 1 handles high-level file transfers.

Once a connection has been established, the user tells the client whether to list the available files, download a file from the server or upload one to it.

4.1 LIST command

To request a list of files available for sharing, the client sends a packet with the CMD_LIST command and requests `filename.txt`.

When the server receives the packet, it calls `crea_lista()` to check whether the `file_disponibili` directory contains any entries. If the directory is empty, the function returns `false`. If it contains one or more files, the function writes their names to `filename.txt` and returns true.

```
bool crea_lista(void){
    const char *target_directory = ``file_disponibili";

    FILE *output_file = fopen("filename.txt", ``w");
    if (output_file == NULL){
        error("Errore␣nell'apertura␣del␣file␣'filename.txt'␣in␣\"crea_lista\"");
    }

    DIR *directory = opendir(target_directory);
    if (directory == NULL) {
        if (errno == ENOENT){
            printf("Non␣sono␣disponibili␣file␣per␣la␣condivisione\n");
            fflush(stdout);
            return false;
        }
        else{
            fclose(output_file);
            error("Errore␣nell'apertura␣della␣directory␣in␣\"crea_lista\"");
        }
    }
}
```

The server checks the return value of `crea_lista()`. If it is `false`, the server sends the client a message stating that no files are available for sharing and waits for another command. If it is `true`, the server calls `spedisci_file()` to send `filename.txt`, then waits for a new request.

```
void spedisci_file(char *client_filename){
    FILE *f = fopen(client_filename, ``r");
    if(f == NULL) error("Errore␣apertura␣file␣in␣\"spedisci_file\"");

    // Buffer for sending the message
    char *buffer = malloc(MAX_PACKET_BODY_LEN);
    if(buffer == NULL)error("Allocazione␣memoria␣fallita␣in␣\"spedisci_file\"");

    char end_flag = 0;
    packet_t *new_packet;
    do{
        memset(buffer, 0, MAX_PACKET_BODY_LEN);
        size_t byte_letti = fread(buffer, sizeof(char), MAX_PACKET_BODY_LEN - 1,
        f);
        if(byte_letti < MAX_PACKET_BODY_LEN - 1){
```

```
            if (feof(f)){
                end_flag = 1;
            }
        else if(ferror(f)){
            error("Errore della funzione \"fread()'' in \"spedisci_file\"");
        }

        new_packet = make_packet(0, 0, 0, 0, CMD_PUT, 0, end_flag, 0, 0,
        client_filename, strlen(client_filename) + 1, buffer, byte_letti + 1, 0,
        0);
        if(handshake == false){
            if(pthread_create(&tid, NULL,apri_connessione, (void*)new_packet)!=0){
                error("Errore nella creazione del thread in \"spedisci_file\"");
            }
            while(handshake == false);
        }
        else{
            send_new_packet(new_packet);
        }
    } while(!feof(f));

    printf("\nUpload di %s completato\n", client_filename);

    fflush(stdout);
    fclose(f);
    free(buffer);
}
```

The client calls ricevi_file() and waits for the server's response.

- If no files are available for sharing, the client prints the error message and returns to the main menu.
- Otherwise, the client temporarily saves the file, displays its contents, deletes it and returns to the main menu.

```
static bool ricevi_file(void){
    // --- Save the file ---
    char end_flag = 0;
    packet_t *packet;
    struct buffer *temp_buff;
    bool first = true;
    FILE *f;
    while(!end_flag){
        while(buffer_list->first == NULL);
        temp_buff = buffer_list->first;
        if(buffer_list->first == buffer_list->last){
            buffer_list->first = NULL;
            buffer_list->last = NULL;
        }else{
            buffer_list->first = buffer_list->first->next;
        }
        packet = temp_buff->packet;
        free(temp_buff);
        if(first == true){
            if(packet->header.error_flag == 1){
```

```
            printf("%s", packet->body);
            fflush(stdout);
            free(packet);
            return false;
        }
    }else{
        f = fopen(packet->filename, ``w");
    }
}
fprintf(f, ``%s",␣packet->body);
fflush(f);

end_flag = packet->header.end_flag;
if(end_flag){
    printf("Download␣di␣%s␣dal␣server␣completato\n", packet->filename);
}
free(packet);

fclose(f);
return true;
}
```

4.2 DOWNLOAD command

To download a file, the client calls `leggi_stringa()` to prompt the user for its name. It then sends a packet with the `CMD_GET` command and sets the filename field accordingly.

When the server receives the packet, it calls `file_presente_nel_server()` to check whether the requested file is in the `file_disponibili` directory.

- If the file is missing, the server sends a message stating that the file is unavailable for sharing and waits for another request.
- Otherwise, the server calls `spedisci_file()`, as described above.

The client calls `ricevi_file()`. It prints the relevant message if an error occurs. Otherwise, it saves the file it receives.

4.3 UPLOAD command

To upload a file to the server, the client calls `leggi_stringa()` to prompt the user for its name. It then calls `spedisci_file()` to send one or more packets with `CMD_PUT` and the appropriate `filename` value.

When the server receives the first packet, it calls `ricevi_file()` and saves the incoming file in `file_disponibili`.

5 Layer 2

Layer 2 implements Selective Repeat. It establishes a reliable connection between the client and server through a **three-way handshake**, handles communication between them and closes the connection through a **four-way handshake**.

5.1 Three-way-handshake

The three-way handshake establishes a reliable connection between the client and server. It ensures that both sides are ready to communicate and synchronises their initial sequence numbers, avoiding stale connections or duplicate packets. The process has three steps.

1. The client randomly selects the sequence number of its first packet, sets the **SYN** flag, sends the packet and starts its timer.

2. The server checks whether the incoming **SYN** is a duplicate. If so, it discards it. Otherwise, it checks whether `thread_list` contains an entry.

   - If the list is empty, the maximum number of fully or half-established connections has been reached. The server rejects the request and notifies the client with a packet whose **RST** flag is set.

   - If the list is not empty, the server removes its first entry, randomly selects a sequence number for the reply and sends it with both **SYN** and **ACK** set.

3. The client replies to the **SYN-ACK** with an **ACK** packet containing the command selected by the user. It allocates `buffer_list`, which serves as the interface between the layers, and `rcv_list`, which holds received packets that have yet to be passed to the upper layer because they arrived out of order.

When the server receives the client's **ACK**, it creates a `thread_struct` and a thread to manage the new connection.

5.2 Client side

5.2.1 Receiving packets

After establishing the connection using `apri_connessione()`, the client calls `thread_routine()` and waits for a packet from the server.

```
while(1){
   packet_t *packet = malloc(MAX_PACKET_LEN);
   ricevi_pacchetto(packet);
```

If the packet contains data, the client sends an **ACK** to the server and discards the packet if it lies outside the receive window.

If its sequence number `n_seq` equals `rcv_base`, the client calls `remove_buffer()` to take the in-order packets from `rcv_list` and append them to `buffer_list`.

If the packet is inside the receive window but is ahead of the next expected packet, the client adds it to `rcv_list`.

```
if(in_window(ntohl(packet->header.n_seq), rcv_base, rcv_limit_window)){
   if(ntohl(packet->header.n_seq) == rcv_base){
      lista_buffer *ret_list = remove_buffer(rcv_list, packet);
      if (buffer_list->first == NULL){// Empty list
```

```
            buffer_list->first = ret_list->first;
            buffer_list->last = ret_list->last;
        }else{
            buffer_list->last->next = ret_list->first;
            buffer_list->last = ret_list->last;
        }
    }else{
        add_buffer(rcv_list, packet);
    }
}else{
    free(packet);
```

If the received packet is an **ACK**, the client calls `handle_ack()` to remove the acknowledged packet from the active timer list.

5.2.2 Sending packets

To send a packet, the client calls `send_new_packet()`. The function waits for space in the send window, sends the packet and starts its timer.

```
void send_new_packet(packet_t *packet){
    packet->header.index = htonl(thread_index);
    packet->header.n_seq = htonl(next_seqnum);

    while(in_window(next_seqnum, send_base, send_limit_window) == false);
    invia_pacchetto(packet);

#ifdef TIMER_FISSO
    timer *new_timer = create_timer(packet, &timeout_fisso);
#elif defined(TIMER_ADATTIVO)
    timer *new_timer = create_timer(packet, &timeout_adattivo);
#endif

    add_timer(new_timer);
    next_seqnum = (next_seqnum + 1) % MAX_MSG_CLUSTER;
```

Note 1. The client uses `ricevi_pacchetto()` and `invia_pacchetto()`, which in turn call the socket functions `recvfrom()` and `sendto()`.

5.3 Server side

The server uses analogous functions, `server_rcv()`, `handle_ack()` and `send_new_packet()`, to receive and transmit packets. They use the `thread_struct` for the corresponding connection, a pointer to which is stored in `thread_array[WORKER_THREADS_NUM]`.

```
// Multithreading
typedef struct _thread_struct{
    struct sockaddr_in *address;          // Destination socket address
    uint32_t thread_index;                // Connection index
    nseq_t rcv_base;
    nseq_t rcv_limit_window;
    nseq_t send_base;
    nseq_t send_limit_window;
    nseq_t next_seqnum;
    struct lista_buffer *rcv_list;
    struct lista_buffer *buffer_list;
```

```
    char array_ack[WINDOW];              // Array of received acknowledgements
    struct timeval *timeout_adattivo;    // Adaptive timer duration
    double estimated_RTT;                // Estimated RTT
    double dev_RTT;                      // RTT variation
    pthread_t tid;                       // Connection thread ID
} thread_struct;
```

5.4 Four-way handshake

When the user chooses to terminate the program, the four-way handshake safely closes the connection between the two hosts while allowing any packets still in flight to be delivered. It

consists of four steps.

1. The client sends a packet with the **FIN** flag set to indicate that it has finished sending data.
2. The server replies with an **ACK** to confirm receipt of the connection-close request.
3. Once the server has finished sending data, it sends its own packet with the **FIN** flag set.
4. The client sends a final **ACK** to confirm closure.

Note 2. To account for possible loss of a FIN packet during connection closure, the client's FIN is placed in `timer_list`. The connection structures are deallocated only after the fourth packet is received.

6 Buffer management

Layer 2 uses two lists.

- `buffer_list` acts as the interface between layers 1 and 2.
- `rcv_list` holds the packets that arrive out of order.

Each list has the following structure.

```
typedef struct lista_buffer{
   buffer *first;
   buffer *last:
}lista_buffer;
```

The list nodes have the following structure.

```
typedef struct buffer{
   packet_t *packet;
   struct buffer *next;
}buffer;
```

The functions `crea_lista_buffer()` and `crea_buffer()` initialise the lists and their nodes.

6.1 The `add_buffer()` function

When a host receives a data packet whose sequence number `n_seq` differs from `rcv_base`, it calls `add_buffer()` to insert the packet into `rcv_list`.

The function first calls `in_window()` to check whether packet is in the receive window. It then checks whether the packet's sequence number is less than `rcv_base`. If it is, the function performs the following steps.

- Traverses all entries in `rcv_list` whose sequence numbers are greater than `rcv_base`.
- Inserts the packet in the appropriate position.

Otherwise, it performs the following step.

- Traverses the entries in `rcv_list` whose sequence numbers are greater than `rcv_base` and inserts the packet in the appropriate position.

6.2 The `remove_buffer()` function

When a host receives a data packet whose `n_seq` equals `rcv_base`, it calls `remove_buffer()` to remove the in-order packets from `rcv_list`. The function also updates `rcv_base` and `rcv_limit_window` to advance the receive window appropriately.

7 Timer management

In Selective Repeat, each transmitted packet has its own **logical timer**. The timer starts when the packet is sent and helps detect losses. If it expires before an acknowledgement is received, the sender retransmits the packet.

The timer list is implemented as `typedef struct lista_timer`. Its entries are defined as follows.

```
typedef struct timer{
   packet_t *packet; // Packet associated with this timer
   struct timeval *start_time; // Time when the timer starts
   struct timeval *curr_timer; // Timer duration
   struct timer *next; // Next list entry
   int count; // Number of retransmissions
}timer;
```

The `timeval` structure is commonly used for timeouts and represents time with microsecond precision.

7.1 Fixed timer implementation

The function `create_timer()` creates a timer node by allocating the necessary memory and initialising its fields.

7.1.1 Adding and removing a timer in `timer_list`

The following function inserts a newly created node into `timer_list`.

```
void add_timer(struct timer *new_timer){
     new_timer->next = NULL;
     if(timer_list.first == NULL){
        timer_list.first = new_timer;

        // Set the timer 'time'
        struct itimerval time;
        time.it_value = *(new_timer->curr_timer);
        // The timer does not repeat
        time.it_interval.tv_sec = 0;
        time.it_interval.tv_usec = 0;

        if(setitimer(ITIMER_REAL, &time, NULL) == -1){
           error("Errore nell'impostazione del timer");
        }
        timer_list.last = new_timer;
     }
     else{
#ifdef TIMER_FISSO
        timer_list.last->next = new_timer;
        timer_list.last = new_timer;
#elif defined(TIMER_ADATTIVO)
        insert_in_order(new_timer);
#endif
     }
}
```

This section describes the `TIMER_FISSO` branch. The adaptive branch is described in Section 7.2.

The function appends the new timer to `timer_list`. If the list is empty, it starts the physical timer by calling `setitimer()`.

Note 3. A computer has a limited number of physical timers, while software can define an arbitrary number of logical timers. The function starts the physical timer for the first node only and keeps the remaining logical timers in ascending order in the list.

To remove a timer from `timer_list`, `remove_timer()` unlinks it and returns it to the caller.

7.1.2 Calculating the remaining time

The following function calculates the time left before a timer expires.

```
static void detect_timer(timer *node, struct itimerval *new_timer){
   if(memset(new_timer, 0, sizeof(struct itimerval)) == NULL){
      error("Errore della funzione \"memset()\"");
   }

   struct timeval ora;
   if(gettimeofday(&ora, NULL) == -1){
      error("Errore della funzione \"gettimeofday()\"");
   }

   long long int timer_da_impostare = tempo(node->curr_timer);
   long long int ora_attuale = tempo(&ora);
   long long int quando_impostato = tempo(node->start_time);
   long long int elapsed_time = ora_attuale - quando_impostato;
   long long int remaining_time = timer_da_impostare - elapsed_time;

   new_timer->it_value.tv_sec = remaining_time / ((long long int)pow(10,6));
   new_timer->it_value.tv_usec = remaining_time % ((long long int)pow(10,6));
   // The timer does not repeat
   new_timer->it_interval.tv_sec = 0;
   new_timer->it_interval.tv_usec = 0;
}
```

7.1.3 Setting the next timer

The function `imposta_timer()` sets the next timer. It first calls `detect_timer()` to determine the time remaining for the first entry in `timer_list`.

If the timer has expired, it removes it using `remove_timer()` and increments its count field. It then checks whether count exceeds `MAX_COUNT`, indicating that the packet has reached its retransmission limit. If the limit has not been reached, the function calls `add_timer()` to put the timer back in `timer_list`. Otherwise, it takes one of the following actions.

- On the client, it notifies the user that the server is unavailable.
- On the server, it checks whether the connection was fully established. If so, it calls `elimina_connessione()` and `delete_thread_struct()` to release the connection resources and terminates the thread managing the connection.

```
new_timer->count++;
thread_struct *t_struct = thread_array[ntohl(new_timer->packet->header.index)];

if(new_timer->count > MAX_COUNT){
```

```
    if(t_struct == NULL){
        // Release the associated resources
    }
    else{
        elimina_connessione(t_struct);
        pthread_kill(t_struct->tid, SIGTERM);
        delete_thread_struct(t_struct->thread_index);
    }
}
```

If the timer has not expired, the function sets it using `setitimer()`.

7.1.4 Handling SIGALRM

Timeout handling lets Selective Repeat retransmit lost packets selectively. Whenever the physical timer expires, **SIGALRM** is delivered to the main thread on the server or to the thread managing the connection on the client.

These threads run `handle_alarm()`, which calls `imposta_timer()`.

7.1.5 Handling acknowledgements

When the host receives an ACK packet, it calls the following function.

```
void handle_ack(nseq_t ack)
    bool first = false;
    if(in_window(ack, send_base, send_limit_window)){
        if(timer_list.first != NULL){
            if(ntohl(timer_list.first->packet->header.n_seq) == ack){
                first = true;
                // Disable the previously set timer
                ............
            }
        }
        timer *ack_timer = remove_timer(ack);
        if(ack_timer != NULL){
            free(ack_timer->packet);
            free(ack_timer->start_time);
            free(ack_timer);
        }

        if((first == true) && (timer_list.first != NULL)){
            imposta_timer();
        }
    }
    // Update array_ack
    ............
}
```

The function checks whether the ACK falls within the window. If it does, the function removes the corresponding timer and frees the allocated memory to prevent leaks. If the ACK refers to the first outstanding packet, the function also updates the `rcv_base` and `rcv_limit_window` values.

Note 4. On the server, this function takes an additional `thread_struct` argument so that each thread can manage its own connection.

The listing omits the update to `array_ack`. The following example illustrates how it works.

The window contains 10 packets with sequence numbers 40 to 49. At first, only packets 42, 43 and 44 have been acknowledged.

```
Inserire numero ack: 41
Numero ack inserito: 41
n_el = 0
Numero elementi eliminati da timer_list: 0
Array_ack:      0       1       1       1       1       0       0       0       0       0
send_base = 40, send_limit_window = 49
[MAIN] Stato della lista nel main:
40 - Remaining time 36 s, 489991 us
45 - Remaining time 36 s, 490009 us
46 - Remaining time 36 s, 490009 us
47 - Remaining time 36 s, 490009 us
48 - Remaining time 36 s, 490008 us
49 - Remaining time 36 s, 490008 us
```

Figure 1. Updating the ACK array

When an ACK is received for packet 41, the corresponding entry in `array_ack` is set to one.

```
Inserire numero ack: 40
Numero ack inserito: 40
[TIMER SERVER] Imposto il timer relativo al numero di sequenza 45
n_el = 5
Numero elementi eliminati da timer_list: 5
Array_ack:      0       0       0       0       0       0       0       0       0       0
send_base = 45, send_limit_window = 4
Inserisco pacchetto 0
Inserisco pacchetto 1
Inserisco pacchetto 2
Inserisco pacchetto 3
Inserisco pacchetto 4
[MAIN] Stato della lista nel main:
45 - Remaining time 31 s, 188303 us
46 - Remaining time 31 s, 188301 us
47 - Remaining time 31 s, 188301 us
48 - Remaining time 31 s, 188299 us
49 - Remaining time 31 s, 188298 us
0 - Remaining time 49 s, 999967 us
1 - Remaining time 49 s, 999969 us
2 - Remaining time 49 s, 999970 us
3 - Remaining time 49 s, 999973 us
4 - Remaining time 49 s, 999974 us
```

Figure 2. Updating the ACK array

When the ACK for packet 40 arrives, its sequence number matches `send_base`. The implementation updates `array_ack`, `send_base` and `send_limit_window` so the send window can slide forward.

7.2 Adaptive timer implementation

The adaptive timer dynamically adjusts how long the sender waits before retransmitting an unacknowledged packet. The timeout is based on the estimated round-trip time (RTT), so it can respond to changing network conditions and avoid premature retransmissions or excessive delays.

The following function calculates the timeout.

```
static void timer_adattivo(long sample_RTT){
   if(estimated_RTT < 0){
```

```
        estimated_RTT = sample_RTT;
        dev_RTT = sample_RTT / 2;
    }else{
        dev_RTT = (1 - BETA) * dev_RTT + BETA * fabs(sample_RTT - estimated_RTT);
        estimated_RTT = (1 - ALPHA) * estimated_RTT + ALPHA * sample_RTT;
    }
    timeout_interval = estimated_RTT + 4*dev_RTT;

    temp = (timeout_interval > TIMER_MIN) ? timeout_interval : TIMER_MIN;

    timeout_adattivo.tv_sec = temp / ((long long int) pow(10,6));
    timeout_adattivo.tv_usec = temp % ((long long int) pow(10,6));
}
```

The timeout is the sum of the estimated RTT, `estimated_RTT`, and four times the RTT deviation estimate, `dev_RTT`.

On the first iteration, the measured RTT and half that value are used to initialise the two variables, respectively.

From the second iteration onwards, the variables are updated using the round-trip time estimation formulas. The values of α and β follow those recommended in "*Computer Networking: A Top-Down Approach*".

Note 5. The adaptive timeout is updated only when an ACK is received for a packet that has never been retransmitted.

```
#ifdef TIMER_ADATTIVO
    if(ack_timer->count == 0){
        // The packet has not been retransmitted
        struct timeval ora;
        if(gettimeofday(&ora, NULL) == -1){
            error("Errore della funzione \"gettimeofday()\"");
        }
        sample_RTT = tempo(&ora) - tempo(ack_timer->start_time);
        timer_adattivo(sample_RTT);
    }
#endif
```

7.2.1 Adding a new timer

The adaptive timer uses the functions from the preceding section, with a few important differences.

When `add_timer()` is called, it inserts the timer at the position in `timer_list` determined by its dynamically assigned expiry time, instead of appending it to the end.

8 Performance analysis

8.1 Varying the packet size

The following simulation shows how transfer time varies with the maximum segment size (MSS).

Consider a 10 MB file transfer with a 10-packet send window, a fixed timeout equal to three times the RTT and a 5% packet loss probability.

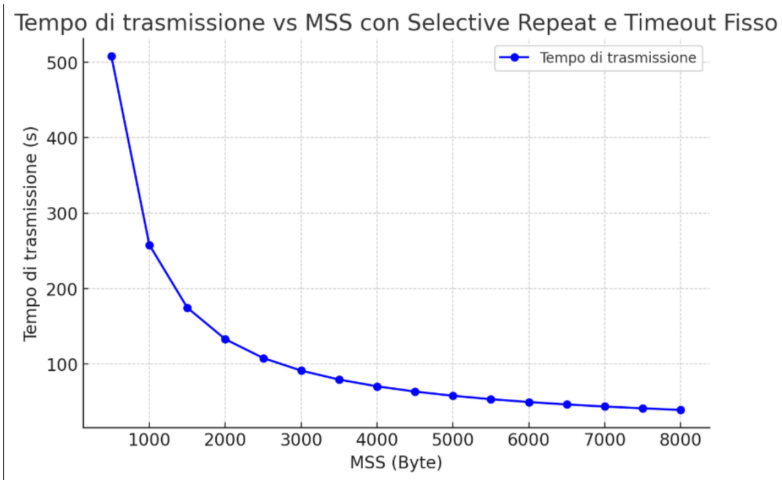


Figure 3. File transfer time in seconds versus MSS in bytes, with a fixed timeout

The following plot shows the result for files of approximately 800 bytes.

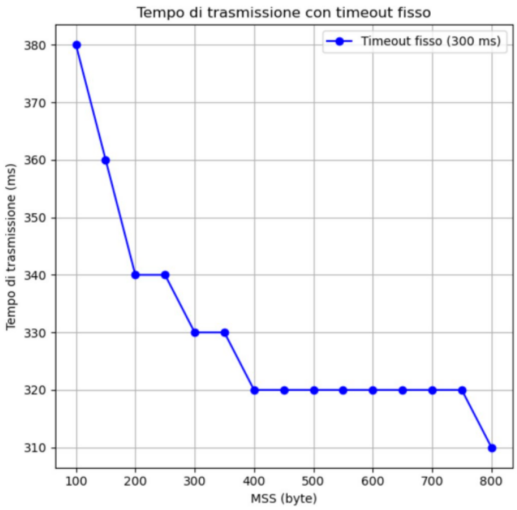


Figure 4. File transfer time in milliseconds versus MSS in bytes, with a fixed timeout

As the MSS increases, fewer packets are required and transfer time decreases. This reduces header and retransmission overhead. Beyond a certain point, timeouts and the finite send window limit further gains.

The next plots show the results with an adaptive timeout.

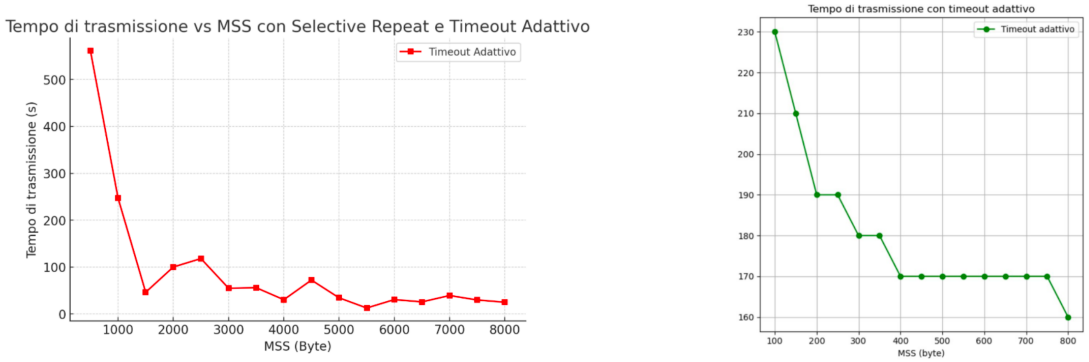


Figure 5. File transfer time versus MSS in bytes, with an adaptive timeout. The larger-file plot uses seconds and the smaller-file plot uses milliseconds.

These plots show how the adaptive timeout responds to changes in network conditions, reducing waiting time and making file transfers more efficient.

Note 6. The fluctuations in Figure 5 depend on how the RTT and its variation are estimated.

8.2 Varying the fixed timeout

The following plot shows how transfer time varies with the fixed timeout.

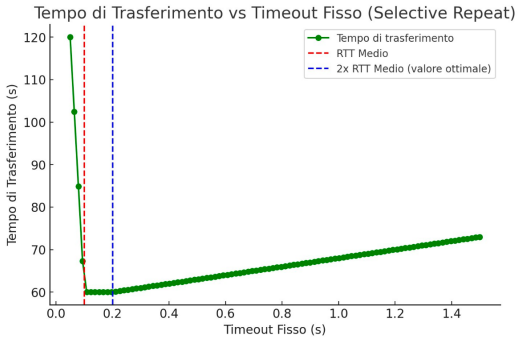


Figure 6. File transfer time in seconds versus the fixed timeout in seconds

- The red line marks the mean RTT. If the timeout falls below this threshold, the protocol retransmits packets unnecessarily because the expected ACKs have not yet arrived.

- The blue line marks the optimal timeout in this experiment, twice the mean RTT. The value is chosen for the network conditions to avoid unnecessary retransmissions and detect packet loss quickly.

8.3 Varying the send window size

We also compare transfer time as the send window size varies.

Consider a 10 MB file transfer with two MSS values, 100 bytes and 4 KB.

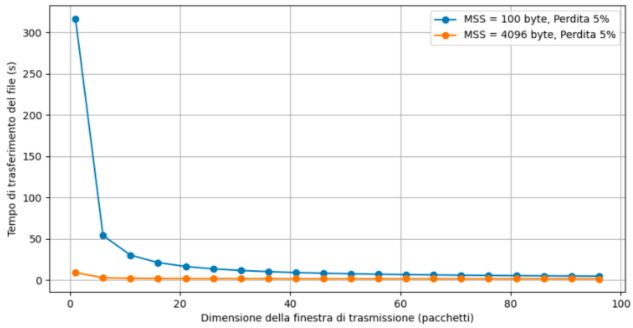


Figure 7. File transfer time in seconds versus send window size in packets

With an MSS of 100 bytes, more packets must be transmitted. Timeouts and acknowledgements have a greater impact, resulting in a substantially longer transfer time.

With an MSS of 4 KB, fewer packets are sent, reducing the number of transmission operations and improving performance.